LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh:

Sebastian Abe Santoso NIM. 2410817210002

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Sebastian Abe Santoso
NIM : 2410817210002

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 8
B. Output Program 18
C. Pembahasan 21
SOAL 2 39
A. Perbedaan XML dan Jetpack Compose 40
B. Kelebihan dan kekurangan dari masing-masing implementasi 40
SOAL Essay 43
A. Fitur yang Menjadi Daya Jual Kotlin 44
B. Test Case Fitur Unggulan Kotlin 45
TAUTAN GITHUB 48
DAFTAR PUSTAKA 49

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Pilihan Persentase Tip 7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan 7
Gambar 4. Screenshot Awal Aplikasi XML 18
Gambar 5. Screenshot Tampilan Pilihan Persentase Tip XML 18
Gambar 6. Screenshot Tampilan Aplikasi Setelah Dijalankan XML 19
Gambar 7. Screenshot Awal Aplikasi Compose 19
Gambar 8. Screenshot Tampilan Pilihan Persentase Tip Compose 20
Gambar 9. Screenshot Tampilan Aplikasi Setelah Dijalankan Compose 20

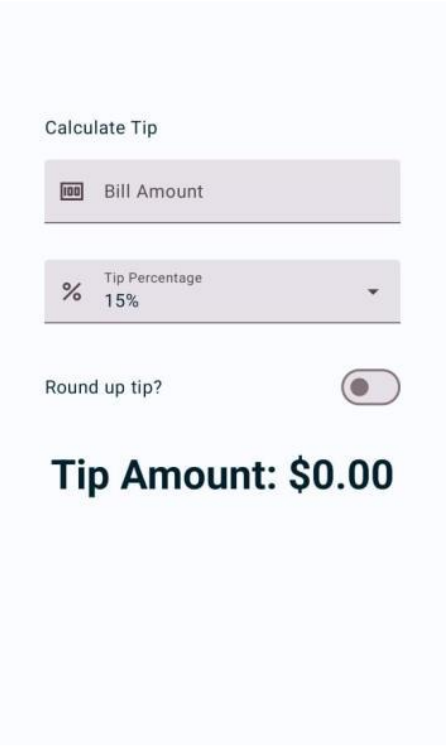
DAFTAR TABEL

Tabel 1. Source Code Main Activity.kt XML 8
Tabel 2. Source Code Main Activity.kt Compose 10
Tabel 3. Source Code activity_main.xml 14
Tabel 4. Test Case 45
Tabel 5. Tidak Menggunakan Lambda 46

SOAL 1

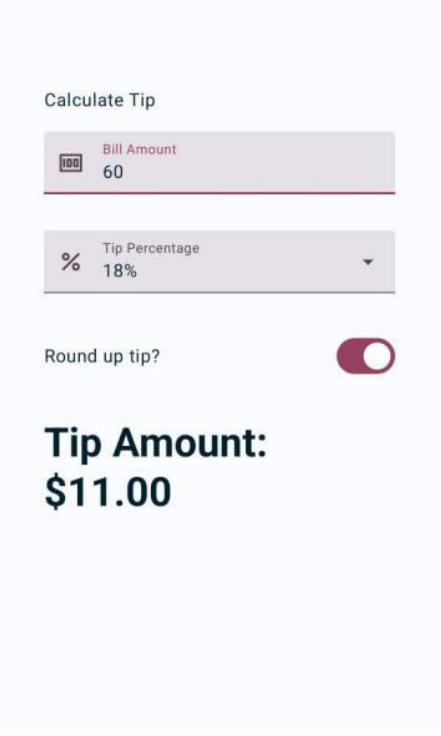
Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi

Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

Tabel 1. Source Code Main Activity.kt XML

No	Main Activity.kt (XML)
1	package com.example.modul2xml
2	
3	import android.os.Bundle
4	import android.widget.ArrayAdapter
5	import android.widget.AutoCompleteTextView
6	import android.widget.TextView
7	import androidx.activity.enableEdgeToEdge
8	import androidx.appcompat.app.AppCompatActivity
9	import androidx.core.view.ViewCompat
10	import androidx.core.view.WindowInsetsCompat
11	import androidx.core.widget.doAfterTextChanged
12	import com.google.android.material.materialswitch.MaterialSwitch
13	import com.google.android.material.textfield.TextInputEditText
14	import java.text.NumberFormat
15	import kotlin.math.ceil
16	import kotlin.toString
17	
18	class MainActivity : AppCompatActivity() {
19	override fun onCreate(savedInstanceState: Bundle?) {
20	super.onCreate(savedInstanceState)
21	*enableEdgeToEdge*()
22	setContentView(R.layout.*activity_main*)
23	
24	val tipOptions = *arrayOf*("15%", "18%", "20%")
25	val adapter = ArrayAdapter(this, android.R.layout.*simple_dropdown_item_1line*, tipOptions)
26	val dropDownMenu = findViewById<AutoCompleteTextView>(R.id.*tipPercentageDropdown*)
27	dropDownMenu.setAdapter(adapter)
28	
29	val billInput = findViewById<TextInputEditText>(R.id.*billInputAmount*)
30	val roundUpSwitch =
31	findViewById<MaterialSwitch>(R.id.*roundUpSwitch*)

```
 billInput.doAfterTextChanged { calculateTip() }

        dropDownMenu.setOnItemClickListener { _, _, _, _ -> calculateTip() }

        roundUpSwitch.setOnCheckedChangeListener { _, _ -> calculateTip() }


ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
    }

    private fun calculateTip () {
        val billInput = findViewById<TextInputEditText>(R.id.billInputAmount)
        val dropDownMenu = findViewById<AutoCompleteTextView>(R.id.tipPercentageDropdown)
        val roundUpSwitch = findViewById<MaterialSwitch>(R.id.roundUpSwitch)
        val tipResult = findViewById<TextView>(R.id.tipAmountTextView)

        val stringInTextField = billInput.text.toString()
        val cost = stringInTextField.toDoubleOrNull()

        if (cost == null) {
            tipResult.text = "Tip Amount: $0.00"
            return
        }

        val tipPercentageString = dropDownMenu.text.toString().replace("%", "")
        val tipPercentage = (tipPercentageString.toDoubleOrNull() ?: 15.0) / 100.0

        var tip = cost * tipPercentage

        if (roundUpSwitch.isChecked) {
```

```
            tip = ceil(tip)
        }

        val formattedTip = NumberFormat.getCurrencyInstance().format(tip)

        tipResult.text = "Tip Amount: $formattedTip"

    }
}
```

Tabel 2. Source Code Main Activity.kt Compose

No	Main Activity.kt (Compose)
1	package com.example.modul2compose
2	
3	import android.annotation.SuppressLint
4	import android.os.Bundle
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.activity.enableEdgeToEdge
8	import androidx.compose.foundation.layout.Arrangement
9	import androidx.compose.foundation.layout.Column
10	import androidx.compose.foundation.layout.PaddingValues
11	import androidx.compose.foundation.layout.Row
12	import androidx.compose.foundation.layout.fillMaxSize
13	import androidx.compose.foundation.layout.fillMaxWidth
14	import androidx.compose.foundation.layout.padding
15	import androidx.compose.foundation.layout.size
16	import androidx.compose.ui.res.painterResource
17	import androidx.compose.material3.DropdownMenuItem
18	import androidx.compose.material3.ExperimentalMaterial3Api
19	import androidx.compose.material3.ExposedDropdownMenuBox
20	import androidx.compose.material3.ExposedDropdownMenuDefaults
21	import androidx.compose.material3.Icon
22	import androidx.compose.material3.MaterialTheme
23	import androidx.compose.material3.MenuAnchorType
24	import androidx.compose.material3.Scaffold
25	import androidx.compose.material3.Switch
26	import androidx.compose.material3.Text
27	import androidx.compose.material3.TextField
28	import androidx.compose.runtime.Composable
29	import androidx.compose.runtime.getValue

```
30 import androidx.compose.runtime.mutableStateOf
31 import androidx.compose.runtime.remember
32 import androidx.compose.runtime.setValue
33 import androidx.compose.ui.Alignment
34 import androidx.compose.ui.Modifier
35 import androidx.compose.ui.text.font.FontWeight
36 import androidx.compose.ui.text.style.TextAlign
37 import androidx.compose.ui.unit.dp
38 import androidx.compose.ui.unit.sp
39 import
   com.example.modul2compose.ui.theme.Modul2ComposeTheme
40
41 class MainActivity : ComponentActivity() {
42     override fun onCreate(savedInstanceState: Bundle?)
   {
43         super.onCreate(savedInstanceState)
44         enableEdgeToEdge()
45         setContent {
46             Modul2ComposeTheme {
47                 MyApp(Modifier, { TipContent(Modifier)
   })
48             }
49         }
50     }
51 }
52
53
54 @Composable
55 fun MyApp(
56     modifier: Modifier = Modifier,
57     tipContent: @Composable (PaddingValues) -> Unit
58 ) {
59     TipScreen(
60         modifier,
61         tipContent,
62     )
63 }
64
65 @Composable
66 fun TipScreen(
67     modifier: Modifier,
68     content: @Composable (PaddingValues) -> Unit,
69 ) {
70     Scaffold(
71         modifier =  modifier.fillMaxSize(),
72     ) { innerPadding ->
73         content(innerPadding)
```

```
    }
}

@SuppressLint("DefaultLocale")
@Composable
fun TipContent(modifier: Modifier = Modifier) {
    Column (
        modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ){
        var amount by remember { mutableStateOf("") }
        var checked by remember { mutableStateOf(false) }
        val amountDouble = amount.toDoubleOrNull()
        var selectedOption by remember { mutableStateOf("15%") }

        Text(
            modifier = Modifier
                .fillMaxWidth(0.9f)
                .padding(top = 20.dp, bottom = 20.dp),
            textAlign = TextAlign.Start,
            text = "Calculate Tip"
        )

        TextField(
            modifier = Modifier.fillMaxWidth(0.9f),
            value = amount,
            onValueChange = { amount = it },
            leadingIcon = {
                Icon(
                    painter = painterResource(id = R.drawable.ic_money),
                    contentDescription = null,
                    modifier = Modifier.size(24.dp)
                )
            },
            label = { Text("Bill Amount") }
        )

        TipPercentageDropdown(
            currentSelection = selectedOption,
            onOptionSelected = { newPercentage -> selectedOption = newPercentage }
        )
```

```

        Row(
            modifier = Modifier.fillMaxWidth(0.9f)
                .padding(bottom = 20.dp, top = 10.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(text = "Round up tip?")

            Switch(
                checked = checked,
                onCheckedChange = { checked = it }
            )
        }

        val percent = selectedOption.replace("%", "")
        var tipAmount = (amountDouble?.times(percent.toDouble())?.div(100)) ?: 0.00
        if (checked) tipAmount = kotlin.math.ceil(tipAmount)

        val formattedTip = String.format("%.2f", tipAmount)
        Text(text = "Tip Amount: $$formattedTip", fontSize = 40.sp, fontWeight = FontWeight.Bold,
            style = MaterialTheme.typography.displaySmall)
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TipPercentageDropdown(currentSelection: String, onOptionSelected: (String) -> Unit) {
    var expanded by remember { mutableStateOf(false) }
    val tipPercentage = listOf("15%", "18%", "20%")

    ExposedDropdownMenuBox(
        expanded = expanded,
        onExpandedChange = { expanded = !expanded },
        modifier = Modifier.padding(top = 20.dp, bottom = 20.dp)
            .fillMaxWidth(0.9f)
```

```
    ) {
        TextField(
            modifier = Modifier.menuAnchor(
                type = MenuAnchorType.PrimaryNotEditable,
                enabled = true
            )
                .fillMaxWidth(),

            label = { Text("Tip Percentage") },
            leadingIcon = { Text("%") },
            value = currentSelection,
            onValueChange = {},
            readOnly = true,
            trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) }
        )

        ExposedDropdownMenu(
            expanded = expanded,
            onDismissRequest = { expanded = false }
        ) {
            tipPercentage.forEach { option ->
                DropdownMenuItem(
                    text = { Text(text = option) },
                    onClick = {
                        onOptionSelected(option)
                        expanded = false
                    },
                    contentPadding = ExposedDropdownMenuDefaults.ItemContentPadding
                )
            }
        }
    }
}
```

Tabel 3. Source Code activity_main.xml

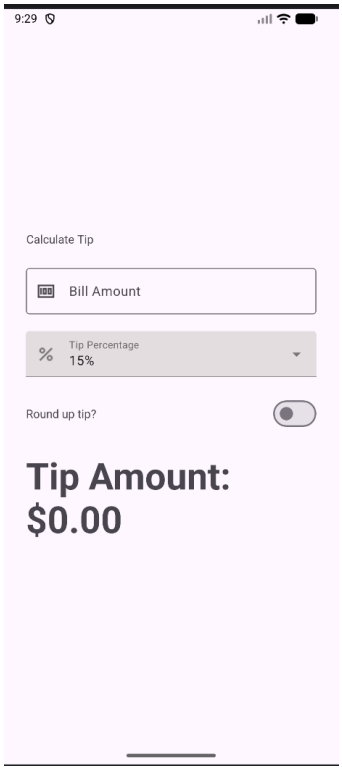
No	activity_main.xml
1	`<?xml version="1.0" encoding="utf-8"?>`
2	`<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"`
3	`    xmlns:app="http://schemas.android.com/apk/res-auto"`

```
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="350dp"
        android:layout_height="wrap_content"
        android:text="Calculate Tip"/>

<com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginStart="30dp"
        android:layout_marginTop="20dp"
        android:layout_marginEnd="30dp"
        android:hint="Bill Amount"
        app:startIconDrawable="@drawable/ic_money">

<com.google.android.material.textfield.TextInputEditTe
xt
            android:id="@+id/billInputAmount"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="numberDecimal"
            android:paddingStart="48dp"
            android:paddingEnd="12dp"/>
</com.google.android.material.textfield.TextInputLayou
t>

<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.MaterialComponents.TextInputLayou
t.FilledBox.ExposedDropdownMenu"
        android:id="@+id/exposedDropDownMenu"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginStart="30dp"
        android:layout_marginTop="20dp"
        android:layout_marginEnd="30dp"
        android:layout_weight="0"
        android:hint="Tip Percentage"
        app:startIconDrawable="@drawable/ic_percent">

```

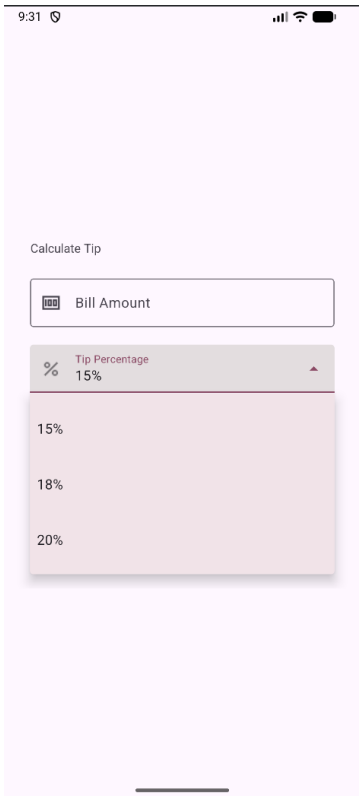
```
  <AutoCompleteTextView
       android:id="@+id/tipPercentageDropdown"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="none"
            android:text="15%" />

</com.google.android.material.textfield.TextInputLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginStart="30dp"
        android:layout_marginTop="20dp"
        android:layout_marginEnd="30dp"
        android:orientation="horizontal">

        <TextView
            android:id="@+id/textView"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Round up tip?"
      app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
         app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

<com.google.android.material.materialswitch.MaterialSwitch
            android:id="@+id/roundUpSwitch"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />
    </LinearLayout>

    <TextView
        android:id="@+id/tipAmountTextView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginStart="30dp"
        android:layout_marginTop="20dp"
        android:layout_marginEnd="30dp"
        android:layout_weight="0"
        android:gravity="left"
        android:text="Tip Amount: $0.00"
```

```
        android:textSize="45sp"
        android:textStyle="bold"/>
</LinearLayout>
```

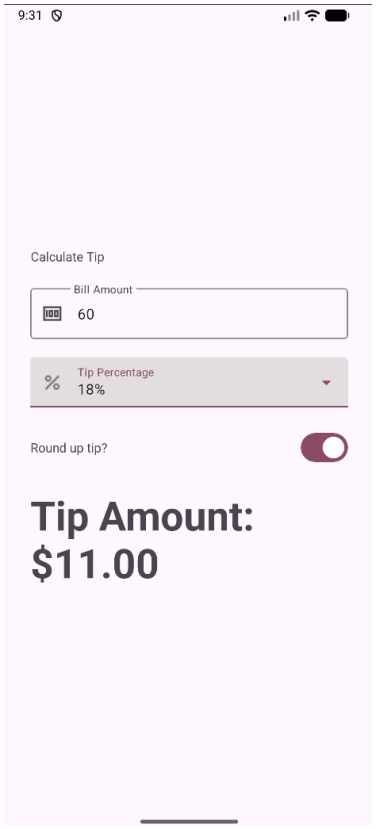
B. Output Program



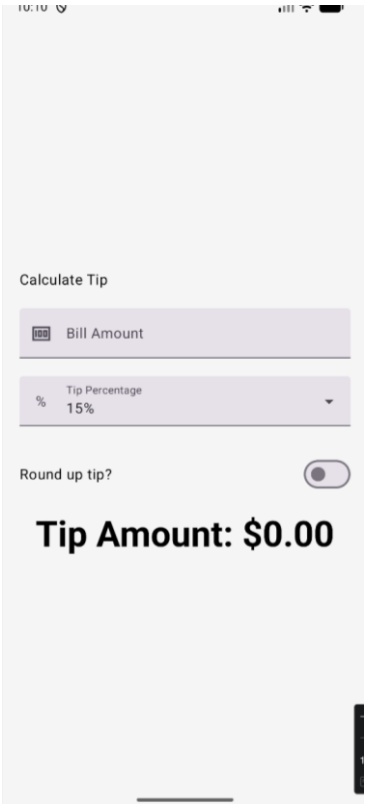
Gambar 4. Screenshot Awal Aplikasi XML



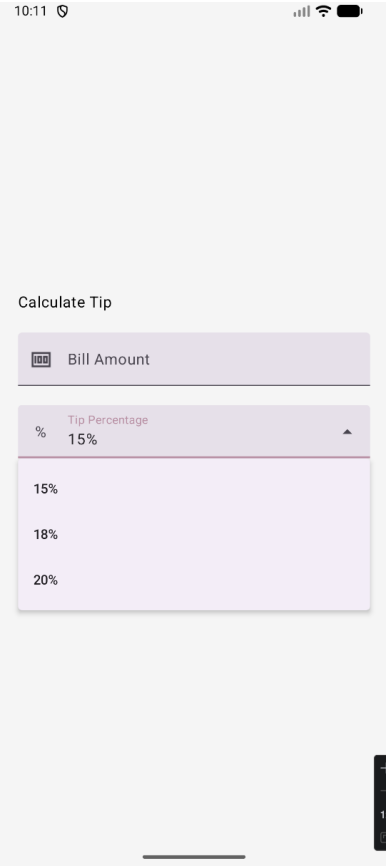
Gambar 5. Screenshot Tampilan Pilihan Persentase Tip XML



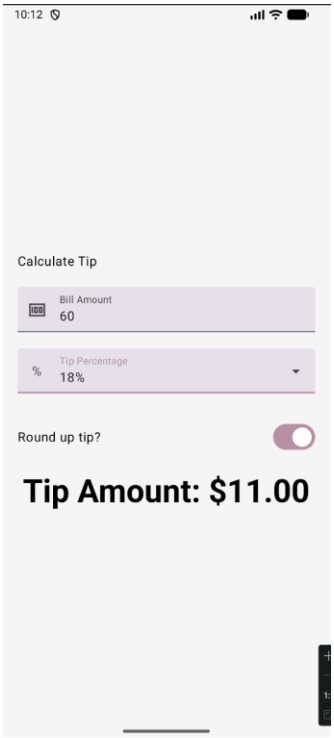
Gambar 6. Screenshot Tampilan Aplikasi Setelah Dijalankan XML



Gambar 7. Screenshot Awal Aplikasi Compose



Gambar 8. Screenshot Tampilan Pilihan Persentase Tip Compose



Gambar 9. Screenshot Tampilan Aplikasi Setelah Dijalankan Compose

C. Pembahasan

MainActivity.kt XML

Pada baris [1] terdapat ‘package com.example.modul2xml’ yang mendefinisikan nama paket aplikasi Android ini, bisa digunakan untuk manajemen file, komponen, ataupun hanya mengorganisasikan kode saja.

Pada baris [3] sampai [16], terdapat beberapa directive import:

- ‘import android.os.Bundle’ (baris 3) digunakan untuk menyimpan *state*, ataupun mengirim data antar *Activity*. Di sini digunakan pada ‘override fun onCreate(savedInstanceState: Bundle?)’
- ‘import android.widget.ArrayAdapter’, ‘AutoCompleteTextView’, dan ‘TextView’ (baris 4 - 6) digunakan untuk komponen UI dan Adapter. ‘ArrayAdapter’ untuk jembatan mengubah data dari *Array* menjadi tampilan yang dapat dipahami oleh UI. ‘TextView’ untuk menampilkan teks di layer, dan ‘AutoCompleteTextView’ untuk memberikan saran otomatis saat pengguna mengetik, selain itu berfungsi sama seperti ’TextView’.
- ‘import androidx.activity.enableEdgeToEdge’ (baris 7) berguna agar tampilan aplikasi dapat memenuhi seluruh layar termasuk area *system bar*.
- ‘import androidx.appcompat.app.AppCompatActivity’ (baris 8) sebagai cetakan untuk *Activity* yang mendukung fitur kompatibilitas, sehingga ‘MainActivity’ atau *Activity* lainnya dapat menggunakan fitur Android terbaru pada versi Android yang lebih lama tanpa *error*.
- ‘import androidx.core.view.ViewCompat’ dan ‘WindowInsetsCompat’ (baris 9 - 10) ViewCompat adalah alat bantu yang memastikan kode dapat dijalankan di berbagai versi android, dan membantu ‘WindowInsetsCompat’ yang berguna untuk menyimpan data lokasi area yang digunakan sistem seperti *status bar* atau *navigation bar*, dan menanganinya agar tidak bentrok dengan aplikasi.
- ‘import androidx.core.widget.doAfterTextChanged’ (baris 11) adalah sebuah *listener* yang berguna untuk mendeteksi perubahan teks secara reaktif pada sebuah kotak *input*.

- ‘import com.google.android.material.materialswitch.MaterialSwitch’ dan ‘TextInputEditText’ (baris 12 - 13) untuk komponen UI Material Design berupa switch dan input teks.
- ‘import java.text.NumberFormat’ (baris 14) untuk memformat angka menjadi format, dalam hal ini digunakan untuk double, format mata uang *dollar*.
- ‘import kotlin.math.ceil’ (baris 15) untuk membulatkan angka ke atas untuk ‘Switch’ yang membulatkan tip ke atas.

Pada baris [18] terdapat deklarasi ‘class MainActivity : AppCompatActivity()’ yang mendefinisikan class utama Activity dan mewarisi ‘AppCompatActivity’.

Pada baris [19] sampai [22], terdapat fungsi ‘onCreate()’ yang merupakan lifecycle method utama Activity:

- ‘super.onCreate(savedInstanceState)’ (baris 20) memanggil berbagai implementasi dari kelas induk untuk menyiapkan infrastruktur untuk Android, dan mengambil *state* jika telah disimpan sebelumnya.
- ‘enableEdgeToEdge()’ (baris 21) mengaktifkan tampilan layar penuh untuk aplikasi.
- ‘setContentView(R.layout.activity_main)’ (baris 22) menghubungkan *Activity* dengan file layout XML ‘activity_main’.

Pada baris [24] sampai [27], terdapat pengaturan komponen *dropdown*:

- ‘val tipOptions = arrayOf("15%", "18%", "20%")’ (baris 24) mendefinisikan *array* pilihan persentase tip, bertipe *val* dan *arrayOf* yang keduanya *immutable*.
- ‘val adapter = ArrayAdapter(...)’ (baris 25) membuat adapter yang menghubungkan data ‘tipOptions’ ke tampilan *dropdown*.
- ‘val dropDownMenu = findViewById<AutoCompleteTextView>(...)’ (baris 26) mengambil referensi komponen dropdown dari main_activity.xml berdasarkan Id.
- ‘dropDownMenu.setAdapter(adapter)’ (baris 27) memasang adapter ke *dropdown* agar list yang dirubah menjadi opsi tampil saat ditekan.

Pada baris [29] sampai [30], terdapat pengambilan referensi dua komponen UI lainnya:

- ‘val billInput’ (baris 29) yang merupakan *input* pada kolom *bill*.
- ‘val roundUpSwitch’ (baris 30) mengacu pada *toggle switch* untuk pembulatan tip ke atas.

Pada baris [32], [34], dan [36], terdapat tiga event *listener* yang masing-masing memanggil ‘fun calculateTip()’ secara otomatis:

- ‘billInput.doAfterTextChanged’ (baris 32) aktif setiap kali teks pada input *bill* berubah.
- ‘dropDownMenu.setOnItemClickListener’ (baris 34) aktif saat pengguna memilih persentase dari *dropdown*.
- ‘roundUpSwitch.setOnCheckedChangeListener’ (baris 36) aktif saat kondisi *switch* berubah.

Pada baris [38] sampai [42], terdapat ‘ViewCompat.setOnApplyWindowInsetsListener’ yang menyesuaikan padding layout utama berdasarkan ukuran *system bar* menggunakan ‘v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)’, sehingga konten aplikasi tidak tertutup komponen sistem.

Pada baris [45] terdapat deklarasi ‘private fun calculateTip()’ sebagai fungsi privat yang hanya dapat dipanggil dari dalam kelas ini, bertugas menghitung dan menampilkan jumlah tip berdasarkan persentase *dropdown* yang telah dipilih.

Pada baris [46] sampai [49], terdapat pengambilan referensi komponen UI yang diperlukan dalam kalkulasi, yaitu ‘billInput’, ‘dropDownMenu’, ‘roundUpSwitch’, dan ‘tipResult’ (TextView untuk menampilkan hasil).

Pada baris [51] sampai [57], terdapat validasi input:

- ‘val stringInTextField’ (baris 51) mengambil teks dari kolom input sebagai String.
- ‘val cost = stringInTextField.toDoubleOrNull()’ (baris 52) mencoba mengkonversi teks menjadi tipe *Double*; jika gagal akan menghasilkan null.
- ‘Blok if (cost == null)’ (baris 54 - 57) menangani kondisi input tidak valid dengan menampilkan "Tip Amount: $0.00" (baris 55) dan menghentikan eksekusi fungsi lebih lanjut menggunakan ‘return’ (baris 56).

Pada baris [59] sampai [62], terdapat proses kalkulasi tip:

- ‘val tipPercentageString’ (baris 59) mengambil teks dari *dropdown* dan menghapus karakter "%" agar bisa dikonversi ke angka untuk kalkulasi.
- ‘val tipPercentage’ (baris 60) mengkonversi string persentase menjadi nilai desimal (misalnya "15" menjadi 0.15), dengan nilai *default* 15.0 apabila konversi gagal.

- ‘var tip = cost * tipPercentage’ (baris 62) menghitung jumlah tip berdasarkan nominal dan persentase yang dipilih.

Pada baris [64] sampai [66], terdapat kondisi ‘if (roundUpSwitch.isChecked)’ yang memeriksa apakah switch pembulatan aktif; jika ya, maka nilai tip dibulatkan ke atas menggunakan fungsi ‘ceil(tip)’ (baris 65).

Pada baris [68] dan [70], terdapat pemformatan dan penampilan hasil:

- ‘val formattedTip = NumberFormat.getCurrencyInstance().format(tip)’ (baris 68) mengformat nilai tip sesuai format mata uang lokal perangkat.
- ‘tipResult.text = "Tip Amount: $formattedTip"’ (baris 70) menampilkan hasil akhir pada ‘TextView’ di layar.

Pada baris [72] dan [73] terdapat penutup kurung kurawal yang mengakhiri fungsi ‘calculateTip()’ dan *class* ‘MainActivity’.

activity_main.xml

Pada baris [1] terdapat deklarasi ‘<?xml version="1.0" encoding="utf-8"?>’ yang merupakan header file XML yang menentukan versi XML digunakan dan encoding UTF-8.

Pada baris [2] sampai [10] terdapat tag pembuka elemen *root* ‘<LinearLayout>’, diubah dari *ConstraintLayout* yang merupakan *default* untuk *Empty Views Activity*. Atribut layoutnyanya:

- ‘xmlns:android, xmlns:app, dan xmlns:tools’ (baris 2 - 4) mendefinisikan *namespace* XML yang diperlukan agar atribut Android dapat dikenali.
- ‘android:id="@+id/main"’(baris 5) memberikan ID pada layout utama sehingga dapat direferensikan di kode Kotlin.
- ‘android:layout_width="match_parent" dan android:layout_height="match_parent"’ (baris 6 - 7) membuat layout mengisi seluruh lebar dan tinggi layar yang dimiliki oleh *parent*.
- ‘android:gravity="center"’ (baris 8) memusatkan seluruh konten di dalam layout secara horizontal dan vertikal, yaitu di tengah.
- ‘android:orientation="vertical"’ (baris 9) mengatur arah susunan elemen anak menjadi vertikal dari atas ke bawah.

- ‘tools:context=".MainActivity"’ (baris 10) menghubungkan layout (activity_main.xml) ini dengan *MainActivity* (MainActivity.kt) untuk keperluan di Android Studio.

Pada baris [12] sampai [15] terdapat komponen ‘TextView’ pertama yang berfungsi sebagai judul halaman:

- ‘android:layout_width="350dp"’ dan ‘android:layout_height="wrap_content"’ (baris 13 - 14) menentukan ukuran TextView untuk selebar 350 dp dan setinggi komponen (membungkus *TextView*).
- ‘android:text="Calculate Tip"’ (baris 15) menampilkan teks judul "Calculate Tip" di atas *TextInputLayout*.

Pada baris [17] sampai [33] terdapat komponen *TextInputLayout* pertama yang berfungsi sebagai wadah *input* nominal tagihan:

- ‘android:layout_marginStart’, ‘android:layout_marginTop’, dan ‘android:layout_marginEnd’ (baris 20 - 22) memberikan jarak 30dp di kiri, 20dp di atas, dan 30dp di kanan.
- ‘android:hint="Bill Amount"’ (baris 23) menampilkan teks pada kolom *input*.
- ‘app:startIconDrawable="@drawable/ic_money"’ (baris 28) menambahkan ikon uang di sisi kiri kolom *input*.
- Di dalamnya, pada baris [26] sampai [32] terdapat ‘TextInputEditText’ dengan ‘android:id="@+id/billInputAmount"’ sebagai ID yang direferensikan di Kotlin, ‘android:inputType="numberDecimal"’ agar keyboard angka desimal muncul saat input, serta ‘android:paddingStart="48dp"’ dan ‘android:paddingEnd="12dp"’ untuk mengatur jarak teks dari tepi kiri (48dp) dan kanan (12dp).

Pada baris [35] sampai [53] terdapat komponen ‘TextInputLayout’ kedua yang berfungsi sebagai yang disebut Google dengan *exposed dropdown* untuk pemilihan persentase tip:

- ‘style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropdownMenu"’ (baris 36) menerapkan gaya Material Design untuk *dropdown* menu.
- ‘android:id="@+id/exposedDropDownMenu"’ (baris 37) memberikan ID pada wadah dropdown.

- ‘android:layout_width=”match_parent’ (baris 38) agar *TextInputLayout* selebar parent, dan ‘android:layout_height=”wrap-content’ (baris 39) agar setinggi *TextInputLayout*nya.
- ‘android:layout_marginstart’, ‘top’, dan ‘end’ untuk memberi margin pada sisi kiri 30dp, atas 20dp, dan kanan 30dp (baris 40-42)
- ‘android:layout_weight=”0”’ untuk menentukan jatah ruang sisa pada parent, yaitu default.
- ‘android:hint="Tip Percentage"’ (baris 44) menampilkan teks pada *dropdown*.
- app:startIconDrawable="@drawable/ic_percent" (baris 45) menambahkan ikon persen di sisi kiri.
- Di dalamnya, pada baris [47] sampai [52] terdapat *AutoCompleteTextView* dengan ‘android:id="@+id/tipPercentageDropdown"’ (baris 48) sebagai ID yang direferensikan di Kotlin, ‘android:layout_width=”match_parent’ (baris 49) agar *TextView* selebar *parent*, dan ‘android:layout_height=”wrap-content’ (baris 50) agar setinggi sendiri ‘android:inputType="none”’ (baris 51) agar keyboard tidak muncul saat ditekan, dan ‘android:text="15%"’ sebagai nilai *default* yang ditampilkan.

Pada baris [56] sampai [79] terdapat LinearLayout kedua yang bersarang, berfungsi menampilkan *label* dan *switch* pembulatan secara berdampingan:

- ‘android:layout_width=”match_parent’ (baris 57) agar TextView selebar parent, dan ‘android:layout_height=”wrap-content’ (baris 58) agar setinggi sendiri, TextView.
- ‘android:layout_marginstart’, ‘top’, dan ‘end’ untuk memberi margin pada sisi kiri 30dp, atas 20dp, dan kanan 30dp (baris 59-61)
- ‘android:orientation="horizontal"’ (baris 62) mengatur susunan elemen *child* menjadi horizontal kiri ke kanan.
- Di dalamnya, pada baris [64] sampai [73] terdapat *TextView* dengan ‘android:id="@+id/textView"’, ‘android:layout_weight="1"’ agar teks mengisi sisa ruang yang tersedia, dan ‘android:text="Round up tip?"’ sebagai label keterangan *switch*.
- Di dalamnya juga, pada baris [75] sampai [78] terdapat *MaterialSwitch* dengan ‘android:id="@+id/roundUpSwitch"’ sebagai ID yang direferensikan di Kotlin,

berfungsi sebagai *toggle* untuk mengaktifkan atau menonaktifkan fitur pembulatan tip ke atas.

Pada baris [81] sampai [92] terdapat *TextView* terakhir yang berfungsi menampilkan hasil perhitungan tip:

- ‘android:id="@+id/tipAmountTextView"’ (baris 82) sebagai ID yang direferensikan di Kotlin untuk memperbarui teks hasil.
- ‘android:layout_width=”match_parent’ (baris 83) agar TextView selebar parent, dan ‘android:layout_height=”wrap-content’ (baris 84) agar setinggi sendiri, TextView.
- ‘android:layout_marginstart’, ‘top’, dan ‘end’ untuk memberi margin pada sisi kiri 30dp, atas 20dp, dan kanan 30dp (baris 85-87
- ‘android:layout_weight=”0”’ (baris 88) untuk menentukan jatah ruang sisa dalam parent, yaitu default.
- ‘android:gravity="left"’ (baris 89) meratakan teks ke sisi kiri.
- ‘android:text="Tip Amount: $0.00"’ (baris 90) sebagai nilai awal yang ditampilkan sebelum kalkulasi dilakukan.
- ‘android:textSize="45sp"’ (baris 91) dan android:textStyle="bold" (baris 92) membuat teks hasil tampil besar dan tebal.

Pada baris [93] terdapat <’/LinearLayout>’ yang menutup elemen *root* dan menandai akhir dari file layout XML.

MainActivity.kt Compose

Pada baris [1] terdapat ‘package com.example.modul2compose’ yang mendefinisikan nama paket aplikasi Android ini, bisa digunakan untuk manajemen file, komponen, ataupun hanya mengorganisasikan kode saja.

Pada baris [3] sampai [39], terdapat beberapa *directive import*:

- ‘import android.annotation.SuppressLint’ (baris 3) digunakan untuk menekan peringatan *lint* tertentu pada kode. Di sini digunakan pada fungsi ‘TipContent()’ dengan anotasi ‘@SuppressLint("DefaultLocale")’ untuk menekan peringatan terkait penggunaan *locale default* pada ‘String.format()’.

- ‘import android.os.Bundle (baris 4)’ digunakan untuk menyimpan *state*, ataupun mengirim data antar *Activity*. Di sini digunakan pada ‘override fun onCreate(savedInstanceState: Bundle?)’.
- ‘import androidx.activity.ComponentActivity’ (baris 5) sebagai cetakan untuk *Activity* berbasis Jetpack Compose, berbeda dengan *AppCompatActivity* yang digunakan pada pendekatan XML, *ComponentActivity* lebih ringan dan dirancang khusus untuk Compose.
- ‘import androidx.activity.compose.setContent’ (baris 6) digunakan sebagai pengganti *setContentView()* pada pendekatan XML, berfungsi menetapkan konten UI menggunakan fungsi *Composable* pengganti file layout di XML.
- ‘import androidx.activity.enableEdgeToEdge’ (baris 7) berguna agar tampilan aplikasi dapat memenuhi seluruh layar termasuk area *system bar*.
- ‘import androidx.compose.foundation.layout.Arrangement’ (baris 8) digunakan untuk mengatur jarak antar elemen dalam layout seperti *Column* atau *Row*, misalnya *Arrangement.Center* untuk memusatkan elemen.
- ‘import androidx.compose.foundation.layout.Column’ (baris 9) digunakan untuk menyusun elemen-elemen UI secara vertikal dari atas ke bawah, setara dengan *LinearLayout* dengan ‘orientation="vertical"’ pada XML.
- ‘import androidx.compose.foundation.layout.PaddingValues’ (baris 10) digunakan untuk mewakili nilai padding pada semua sisi, digunakan sebagai tipe parameter *innerPadding* yang diberikan oleh *Scaffold*.
- ‘import androidx.compose.foundation.layout.Row’ (baris 11) digunakan untuk menyusun elemen-elemen UI secara horizontal dari kiri ke kanan, setara dengan *LinearLayout* dengan ‘orientation="horizontal"’ pada XML.
- ‘import androidx.compose.foundation.layout.fillMaxSize’ (baris 12) adalah modifier yang membuat komponen mengisi seluruh ukuran ruang *parent* yang tersedia.
- ‘import androidx.compose.foundation.layout.fillMaxWidth’ (baris 13) adalah *modifier* yang membuat komponen mengisi seluruh lebar ruang *parent* yang tersedia, dapat dikombinasikan dengan nilai pecahan, contohnya dalam kode adalah ‘fillMaxWidth(0.9f)’ untuk mengisi 90% lebar layar.

- ‘import androidx.compose.foundation.layout.padding’ (baris 14) adalah *modifier* yang memberikan jarak di sekitar komponen, dapat diatur per sisi maupun sekaligus.
- ‘import androidx.compose.foundation.layout.size’ (baris 15) adalah *modifier* yang mengatur ukuran lebar dan tinggi komponen sekaligus menjadi nilai yang sama.
- ‘import androidx.compose.ui.res.painterResource’ (baris 16) digunakan untuk memuat *resource* gambar atau ikon dari folder *drawable* berdasarkan ID resource-nya.
- ‘import androidx.compose.material3.DropdownMenuItem’ (baris 17) digunakan sebagai komponen untuk setiap item pilihan yang ditampilkan di dalam *dropdown menu*.
- ‘import androidx.compose.material3.ExperimentalMaterial3Api’ (baris 18) adalah anotasi yang diperlukan karena *ExposedDropdownMenuBox* masih bersifat eksperimental pada *Material3*, harus digunakan bersama *@OptIn* untuk menyatakan persetujuan penggunaan API eksperimental tersebut.
- ‘import androidx.compose.material3.ExposedDropdownMenuBox’ (baris 19) digunakan sebagai wadah utama komponen *dropdown* yang mengikuti desain *Material3*, mengelola kondisi buka/tutup *dropdown*.
- ‘import androidx.compose.material3.ExposedDropdownMenuDefaults’ (baris 20) menyediakan nilai dan komponen bawaan untuk *ExposedDropdownMenuBox* seperti ‘TrailingIcon’ dan ‘ItemContentPadding’.
- ‘import androidx.compose.material3.Icon’ (baris 21) digunakan untuk menampilkan ikon dari *resource drawable* atau *icon vector* di dalam komponen UI.
- ‘import androidx.compose.material3.MaterialTheme’ (baris 22) menyediakan akses ke sistem desain *Material3* seperti skema warna, tipografi, dan bentuk yang konsisten di seluruh aplikasi.
- ‘import androidx.compose.material3.MenuAnchorType’ (baris 23) digunakan untuk menentukan jenis *anchor* pada *menuAnchor() modifier*, yaitu titik yang menghubungkan *TextField* sebagai untuk dropdown.
- ‘import androidx.compose.material3.Scaffold’ (baris 24) digunakan sebagai kerangka layar *Material Design* yang menyediakan struktur layout standar seperti slot untuk *topBar*, *bottomBar*, dan konten utama beserta *innerPadding*nya.

- ‘import androidx.compose.material3.Switch’ (baris 25) digunakan sebagai komponen *toggle Material Design* yang setara dengan *MaterialSwitch* pada XML.
- ‘import androidx.compose.material3.Text’ (baris 26) digunakan untuk menampilkan teks di layar, setara dengan *TextView* pada XML namun ditulis sepenuhnya dalam kode Kotlin.
- ‘import androidx.compose.material3.TextField’ (baris 27) digunakan sebagai komponen *input* teks *Material Design*, setara dengan *TextInputEditText* pada XML.
- ‘import androidx.compose.runtime.Composable’ (baris 28) adalah anotasi wajib yang menandai sebuah fungsi sebagai fungsi *Composable* sehingga dapat merender UI dan dipanggil dari fungsi *Composable* lainnya.
- ‘import androidx.compose.runtime.getValue’ dan ‘setValue’ (baris 29 dan 32) adalah operator yang memungkinkan penggunaan sintaks *by* untuk membaca dan menulis *state* Compose secara lebih ringkas.
- ‘import androidx.compose.runtime.mutableStateOf’ (baris 30) digunakan untuk membuat objek *state* yang dapat dipantau perubahannya oleh Compose, sehingga UI akan otomatis diperbarui ketika nilainya berubah.
- ‘import androidx.compose.runtime.remember’ (baris 31) digunakan untuk menyimpan nilai *state* di antara rekomposisi, tanpa *remember* maka *state* akan direset setiap kali fungsi Composable dieksekusi ulang.
- ‘import androidx.compose.ui.Alignment’ (baris 33) digunakan untuk mengatur elemen dalam layout, misalnya *Alignment.CenterHorizontally* untuk memusatkan elemen secara horizontal.
- ‘import androidx.compose.ui.Modifier’ (baris 34) adalah objek yang digunakan untuk memodifikasi tampilan dan perilaku komponen Compose seperti ukuran, padding, dan alignment.
- ‘import androidx.compose.ui.text.font.FontWeight’ (baris 35) digunakan untuk mengatur ketebalan font teks, misalnya ‘FontWeight.Bold’ untuk teks tebal.
- ‘import androidx.compose.ui.text.style.TextAlign’ (baris 36) digunakan untuk mengatur perataan teks di dalam komponen *Text*, misalnya ‘TextAlign.Start’ untuk meratakan teks ke kiri.

- ‘import androidx.compose.ui.unit.dp’ (baris 37) adalah satuan dimensi *density-independent pixel* yang digunakan untuk ukuran layout agar konsisten di berbagai ukuran layar.
- ‘import androidx.compose.ui.unit.sp’ (baris 38) adalah satuan *dimensi scale-independent pixel* yang digunakan khusus untuk ukuran teks.
- ‘import com.example.modul2compose.ui.theme.Modul2ComposeTheme’ (baris 39) mengimpor tema kustom aplikasi yang mendefinisikan skema warna dan tipografi yang akan diterapkan secara global.

Pada baris [41] terdapat deklarasi ‘class MainActivity : ComponentActivity()’ yang mendefinisikan kelas utama *Activity* dan mewarisi *ComponentActivity* sebagai kelas dasar yang dirancang khusus untuk Jetpack Compose.

Pada baris [42] sampai [51], terdapat fungsi ‘onCreate()’ yang merupakan *lifecycle method* utama *Activity*:

- ‘super.onCreate(savedInstanceState)’ (baris 43) memanggil berbagai implementasi dari kelas induk untuk menyiapkan infrastruktur Android dan mengambil *state* jika telah disimpan sebelumnya.
- ‘enableEdgeToEdge()’ (baris 44) mengaktifkan tampilan layar penuh untuk aplikasi.
- ‘setContent {...}’ (baris 45 - 49) menetapkan konten UI menggunakan Compose sebagai pengganti ‘setContentView()’ pada pendekatan XML. Di dalamnya memanggil *Modul2ComposeTheme* (baris 46) untuk menerapkan tema kustom secara menyeluruh, kemudian memanggil ‘MyApp(Modifier, { TipContent(Modifier) })’ (baris 47) sebagai fungsi Composable utama dengan meneruskan ‘TipContent’ sebagai lambda konten.

Pada baris [54] sampai [63], terdapat fungsi ‘Composable MyApp()’:

- Dideklarasikan dengan anotasi @Composable (baris 53) sehingga dapat merender UI dan dipanggil dari fungsi Composable lainnya.
- Menerima dua parameter yaitu ‘modifier: Modifier = Modifier’ (baris 56) dengan nilai *default Modifier* kosong, dan ‘tipContent: @Composable (PaddingValues) -> Unit’ (baris 57) berupa *lambda* Composable yang menerima *PaddingValues* sebagai parameter.

- Fungsi ini bertugas sebagai jembatan yang hanya meneruskan kedua parameter tersebut ke fungsi ‘TipScreen()’ (baris 59 - 62), mengikuti pola arsitektur Compose untuk memisahkan tanggung jawab antar Composable.

Pada baris [65] sampai [75], terdapat fungsi Composable TipScreen():

- Dideklarasikan dengan anotasi @Composable (baris 65) sehingga dapat merender UI dan dipanggil dari fungsi Composable lainnya.
- Menerima dua parameter yaitu ‘modifier: Modifier’ (baris 67) dan ‘content: @Composable (PaddingValues) -> Unit’ (baris 68) berupa *lambda* Composable yang menerima *PaddingValues* sebagai parameter.
- Di dalamnya menggunakan *Scaffold* (baris 70 - 74) sebagai kerangka layar *Material Design* dengan ‘modifier = modifier.fillMaxSize()’ agar mengisi seluruh layar. *Scaffold* menyediakan *lambda* ‘{ innerPadding -> content(innerPadding) }’ (baris 72 - 73) yang meneruskan nilai padding sistem ke fungsi content sehingga konten di dalamnya tidak tertutup oleh *system bar*.

Pada baris [77] sampai [78], terdapat dua anotasi sebelum deklarasi fungsi ‘TipContent():’

- ‘@SuppressLint("DefaultLocale")’ (baris 77) menekan peringatan *lint* yang muncul akibat penggunaan ‘String.format("%.2f", tipAmount)’ tanpa menyertakan parameter *locale* secara eksplisit.
- @Composable (baris 78) menandai fungsi ini sebagai fungsi Composable yang dapat merender UI.

Pada baris [79] sampai [84], terdapat deklarasi fungs’i TipContent()’ dengan parameter ‘modifier: Modifier = Modifier’ yang menggunakan *Column* sebagai layout utama:

- ‘modifier.fillMaxSize()’ (baris 80) membuat Column mengisi seluruh ukuran layar.
- ‘horizontalAlignment = Alignment.CenterHorizontally’ (baris 81) memusatkan seluruh elemen *child* secara horizontal.
- ‘verticalArrangement = Arrangement.Center’ (baris 82) memusatkan seluruh elemen anak secara vertikal sehingga konten tampil di tengah layar.

Pada baris [85] sampai [88], terdapat deklarasi 4 *state* yang dikelola dengan *remember* dan *mutableStateOf*:

- ‘var amount by remember { mutableStateOf("") }’ (baris 85) menyimpan teks *input* nominal bill dengan nilai awal string kosong. Penggunaan *by* memungkinkan akses dan perubahan nilai secara langsung tanpa ’.value’.
- ‘var checked by remember { mutableStateOf(false) }’ (baris 86) menyimpan kondisi *switch* pembulatan dengan nilai awal *false*, artinya fitur pembulatan nonaktif saat pertama kali aplikasi dibuka.
- ‘val amountDouble = amount.toDoubleOrNull()’ (baris 87) mengkonversi nilai *amount* menjadi *Double* secara langsung tanpa *remember* karena nilainya diturunkan dari *amount* yang sudah berupa *state*, menghasilkan null jika teks tidak dapat dikonversi menjadi angka.
- ‘var selectedOption by remember { mutableStateOf("15%") }’ (baris 88) menyimpan pilihan persentase tip yang sedang aktif dengan nilai awal "15%" sebagai pilihan default.

Pada baris [90] sampai [96], terdapat komponen Text pertama yang berfungsi sebagai judul halaman:

- ‘modifier = Modifier.fillMaxWidth(0.9f)’ (baris 91 - 92) membuat teks mengisi 90% lebar layar sehingga memiliki margin kiri dan kanan secara simetris.
- ‘.padding(top = 20.dp, bottom = 20.dp)’ (baris 92) memberikan jarak 20dp di atas dan bawah judul.
- ‘textAlign = TextAlign.Start’ (baris 93) meratakan teks ke sisi kiri.
- ‘text = "Calculate Tip"’ (baris 95) menampilkan teks judul halaman.

Pada baris [98] sampai [110], terdapat komponen *TextField* untuk input nominal tagihan:

- ‘modifier = Modifier.fillMaxWidth(0.9f)’ (baris 99) membuat *TextField* mengisi 90% lebar layar, konsisten dengan lebar judul di atasnya.
- ‘value = amount’ (baris 100) menghubungkan *TextField* dengan *state amount* sebagai sumber kebenaran tunggal nilai yang ditampilkan.
- ‘onValueChange = { amount = it }’ (baris 101) memperbarui *state amount* setiap kali pengguna mengetik, yang kemudian memicu rekomposisi sehingga UI selalu sinkron dengan *input*.
- ‘leadingIcon’ (baris 102 - 108) menampilkan ikon uang dari *resource drawable ic_money* berukuran 24dp di sisi kiri kolom *input* menggunakan komponen *Icon*.

- ‘label = { Text("Bill Amount") }’ (baris 109) menampilkan label placeholder yang akan naik ke atas saat TextField sedang aktif diisi.

Pada baris [112] sampai [115], terdapat pemanggilan fungsi Composable ‘TipPercentageDropdown()’ dengan 2 parameter:

- ‘currentSelection = selectedOption’ (baris 113) meneruskan nilai *state* selectedOption agar *dropdown* menampilkan pilihan yang sedang aktif.
- ‘onOptionSelected = { newPercentage -> selectedOption = newPercentage }’ (baris 114) adalah *lambda callback* yang memperbarui *state selectedOption* ketika pengguna memilih opsi baru, mengikuti pola *state hoisting* Compose.

Pada baris [117] sampai [129], terdapat komponen *Row* yang menampilkan label dan *switch* secara berdampingan:

- ‘modifier = Modifier.fillMaxWidth(0.9f)’ (baris 118) membuat *Row* mengisi 90% lebar layar.
- ‘.padding(bottom = 20.dp, top = 10.dp)’ (baris 119) memberikan jarak 10dp di atas dan 20dp di bawah *Row*.
- ‘horizontalArrangement = Arrangement.SpaceBetween’ (baris 120) mendorong label teks ke sisi kiri dan switch ke sisi kanan dengan jarak maksimal di antara keduanya.
- ‘verticalAlignment = Alignment.CenterVertically’ (baris 121) memastikan label dan *switch* sejajar secara vertikal di tengah *Row*.
- ‘Text(text = "Round up tip?")’ (baris 123) menampilkan label keterangan fungsi *switch*.
- *Switch* (baris 125 - 128) dengan ‘checked = checked’ menampilkan kondisi *toggle* sesuai *state*, dan ‘onCheckedChange = { checked = it }’ memperbarui *state checked* setiap kali pengguna menekan *switch*.

Pada baris [131] sampai [133], terdapat proses kalkulasi tip yang dihitung ulang secara otomatis setiap kali state amount, selectedOption, atau checked berubah karena berada langsung di dalam fungsi Composable:

- ‘val percent = selectedOption.replace("%", "")’ (baris 131) mengambil angka persentase murni dari *selectedOption* dengan menghapus karakter "%" agar dapat dikonversi ke angka untuk kalkulasi.

- ‘var tipAmount = (amountDouble?.times(percent.toDouble())?.div(100)) ?: 0.00’ (baris 132) menghitung jumlah tip menggunakan operator *safe* ‘call ?.times()’ untuk perkalian dan ‘?.div(100)’ untuk konversi persentase ke desimal, dengan nilai *default* 0.00 menggunakan operator *Elvis* ?: jika *amountDouble* bernilai null.
- ‘if (checked) tipAmount = kotlin.math.ceil(tipAmount)’ (baris 133) membulatkan nilai *tipAmount* ke atas menggunakan ‘kotlin.math.ceil()’ apabila *state checked* bernilai *true*.

Pada baris [135] sampai [137], terdapat pemformatan dan penampilan hasil akhir:

- ‘val formattedTip = String.format("%.2f", tipAmount)’ (baris 135) memformat nilai *tipAmount* menjadi *string* dengan tepat dua angka desimal, berbeda dengan XML yang menggunakan ‘NumberFormat.getCurrencyInstance()’ untuk format mata uang lokal.
- Komponen *Text* (baris 136 - 137) menampilkan hasil dengan ‘text = "Tip Amount: $$formattedTip"’ menggunakan *string template* Kotlin, ‘fontSize = 40.sp’ untuk ukuran teks besar, ‘fontWeight = FontWeight.Bold’ untuk teks tebal, dan ‘style = MaterialTheme.typography.displaySmall’ untuk menerapkan gaya tipografi *Material Design*.

Pada baris [138] dan [139] terdapat penutup kurung kurawal } yang mengakhiri blok *Column* sekaligus mengakhiri fungsi *TipContent(),* dan kurung kurawal } yang mengakhiri fungsi TipContent() secara keseluruhan.

Pada baris [141] sampai [143], terdapat dua anotasi dan deklarasi fungsi TipPercentageDropdown():

- ‘@OptIn(ExperimentalMaterial3Api::class)’ (baris 141) menyatakan persetujuan eksplisit penggunaan API Material3 yang masih bersifat eksperimental, khususnya *ExposedDropdownMenuBox*. Tanpa anotasi ini, kompiler akan menampilkan error karena API tersebut belum dinyatakan stabil oleh Google.
- *@Composable* (baris 142) menandai fungsi ini sebagai fungsi Composable sehingga dapat merender UI dan dipanggil dari fungsi Composable lainnya.
- ‘fun TipPercentageDropdown(currentSelection: String, onOptionSelected: (String) -> Unit)’ (baris 143) mendeklarasikan fungsi dengan 2 parameter yaitu *currentSelection* bertipe *String* yang menerima nilai pilihan aktif dari *state*

selectedOption di *TipContent*, dan *onOptionSelected* bertipe *lambda* ‘(String) -> Unit’ sebagai *callback* yang dipanggil saat pengguna memilih opsi baru, mengikuti pola *state hoisting* Compose di mana state dikelola di level atas dan hanya diturunkan ke komponen *child* sebagai parameter.

Pada baris [144] sampai [145], terdapat deklarasi 2 variabel lokal:

- ‘var expanded by remember { mutableStateOf(false) }’ (baris 144) menyimpan kondisi buka/tutup *dropdown* dengan nilai awal *false* artinya *dropdown* tertutup saat pertama kali muncul. Penggunaan *remember* memastikan nilai *expanded* tidak direset setiap kali terjadi rekomposisi.
- ‘val tipPercentage = listOf("15%", "18%", "20%")’ (baris 145) mendefinisikan daftar pilihan persentase tip sebagai *List* yang bersifat *immutable* menggunakan *listOf*, berbeda dengan kode saya XML yang menggunakan *arrayOf* karena *listOf* lebih idiomatis dalam Kotlin dan Compose.

Pada baris [147] sampai [151], terdapat pembuka komponen ExposedDropdownMenuBox beserta atribut-atributnya:

- ‘expanded = expanded’ (baris 148) menghubungkan kondisi buka/tutup komponen dengan *state expanded* sehingga tampilan selalu sinkron dengan nilai *state*.
- ‘onExpandedChange = { expanded = !expanded }’ (baris 149) membalik nilai *state expanded* setiap kali pengguna menekan area *TextField*, sehingga *dropdown* bergantian terbuka dan tertutup.
- ‘modifier = Modifier.padding(top = 20.dp, bottom = 20.dp).fillMaxWidth(0.9f)' (baris 150 - 151) memberikan jarak 20dp di atas dan bawah komponen sekaligus membatasi lebarnya menjadi 90% layar agar konsisten dengan komponen lainnya.

Pada baris [153] sampai [166], terdapat komponen *TextField* di dalam *ExposedDropdownMenuBox* yang berfungsi sebagai tampilan utama *dropdown* saat tertutup:

- ‘modifier = Modifier.menuAnchor(type = MenuAnchorType.PrimaryNotEditable, enabled = true)’ (baris 154 - 157) menandai *TextField* ini sebagai titik terjadi *dropdown*. ‘MenuAnchorType.PrimaryNotEditable’ digunakan karena *TextField* ini tidak dimaksudkan untuk diketik, hanya sebagai pemicu dropdown.
- ‘.fillMaxWidth()’ (baris 158) membuat *TextField* mengisi seluruh lebar *ExposedDropdownMenuBox* sebagai wadahnya.

- ‘label = { Text("Tip Percentage") }’ (baris 159) menampilkan label yang akan naik ke atas saat dropdown aktif.
- ‘leadingIcon = { Text("%") }’ (baris 160) menampilkan simbol persen di sisi kiri *TextField* sebagai pengganti ikon *drawable*, berbeda dengan XML yang menggunakan ‘app:startIconDrawable="@drawable/ic_percent"’.
- ‘value = currentSelection’ (baris 161) menampilkan nilai pilihan aktif yang diterima dari parameter fungsi.
- ‘onValueChange = {}’ (baris 162) dibiarkan kosong karena *TextField* ini tidak menerima input langsung dari pengguna.
- ‘readOnly = true’ (baris 163) memastikan keyboard tidak muncul saat *TextField* ditekan sehingga interaksi hanya melalui *dropdown*.
- ‘trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) }’ (baris 165) menampilkan ikon panah di sisi kanan yang secara otomatis berotasi ke atas saat dropdown terbuka dan ke bawah saat tertutup sesuai nilai *state expanded*.

Pada baris [168] sampai [181], terdapat komponen ‘ExposedDropdownMenu’ yang merupakan bagian *dropdown* yang muncul saat *expanded* bernilai *true*:

- ‘expanded = expanded’ (baris 169) mengontrol visibilitas menu berdasarkan *state* yang sama dengan *ExposedDropdownMenuBox*.
- ‘onDismissRequest = { expanded = false }’ (baris 170) menutup *dropdown* secara *otomatis* dengan mengatur *expanded* menjadi *false* saat pengguna menekan area di luar *dropdown*.
- ‘tipPercentage.forEach { option -> }’ (baris 172 - 180) melakukan iterasi pada seluruh elemen dalam ‘tipPercentage’ dan membuat satu *DropdownMenuItem* untuk setiap opsi.
- ‘DropdownMenuItem’ (baris 173 - 179) pada setiap iterasi memiliki tiga parameter yaitu ‘text = { Text(text = option) }’ (baris 174) untuk menampilkan teks opsi, ‘onClick’ (baris 175 - 178) yang memanggil ‘onOptionSelected(option)’ untuk meneruskan pilihan ke *state* ‘selectedOption’ di ‘TipContent’ sekaligus menutup *dropdown* dengan ‘expanded = false’, dan ‘contentPadding =

ExposedDropdownMenuDefaults.ItemContentPadding' (baris 179) untuk menerapkan padding standar *Material Design* pada setiap item.

Pada baris [182] sampai [184], terdapat tiga penutup kurung kurawal } yang masing-masing mengakhiri blok *ExposedDropdownMenu* (baris 182), blok *ExposedDropdownMenuBox* (baris 183), dan baris [184] menandai akhir dari keseluruhan fungsi 'TipPercentageDropdown' sekaligus akhir file 'MainActivity.kt'.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Perbedaan XML dan Jetpack Compose

Untuk perbedaan antara XML dan Compose sudah saya jelaskan sebelumnya dalam baris baris penjelasan dalam kode, namun jika secara lebih spesifik saya ulangi:

Dalam mendapatkan UI, XML memisahkan tampilan (activity_main.xml) menggunakan tag seperti '<LinearLayout>' kemudian dihubungkan ke kode Kotlin melalui 'setContentView' dan direferensikan melalui 'findViewById()' atau *View Binding* untuk terhubung kepada Kotlin. Dalam Compose, UI didefinisikan sepenuhnya dalam kode Kotlin menggunakan fungsi fungsi seperti *Column*, tidak diperlukan file terpisah sehingga tampilan dan logika dapat berada pada file yang sama.

Dalam cara mengelola state, XML perubahan data harus diperbarui secara manual ke UI, misalnya pengguna mengubah input, *listener* 'doAfterTextChanged' dipanggil, kemudian 'calculateTip()' dieksekusi untuk memperbarui UI. Dalam Compose, *state* dikelola menggunakan *remember* dan *mutableStateOf*. Saat nilai seperti 'amount' berubah, Compose secara otomatis melakukan rekomposisi dan memperbarui seluruh UI yang tergantung pada nilai tersebut tanpa keharusan manual.

Dalam cara mengimplementasikan *dropdown*, XML menggunakan *AutoCompleteTextView* yang dibungkus dalam *TextInputLayout* dengan style *ExposedDropdownMenu*, lalu dihubungkan ke sumber data melalui *ArrayAdapter* untuk menampilkan daftar pilihan. Dalam Compose, dropdown dibuat menggunakan *ExposedDropdownMenuBox* yang membungkus *TextField* dan *ExposedDropdownMenu*, dengan *state* seperti *expanded* yang mengatur kondisi terbuka atau tertutup secara reaktif pada UI.

Terakhir, dalam cara menangani padding sistem, XML mengaturnya secara manual menggunakan *ViewCompat.setOnApplyWindowInsetsListener* untuk membaca nilai *systemBars*, lalu menerapkannya ke layout melalui *v.setPadding()* agar tampilan tidak tertutup oleh sistem UI. Dalam Compose, padding sistem ditangani oleh *Scaffold* yang menyediakan *innerPadding* dan langsung diteruskan ke konten di dalamnya, sehingga penyesuaian terhadap *insets* berlangsung otomatis tanpa perlu pengaturan manual.

B. Kelebihan dan kekurangan dari masing-masing implementasi

- XML:

Kelebihan:

1. Tampilan UI dapat dilihat secara langsung di Android Studio melalui fitur Layout Editor tanpa perlu menjalankan aplikasi.
2. Lebih familiar bagi developer Android yang sudah lama menggunakan pendekatan tradisional.
3. Pemisahan antara tampilan dan logika yang jelas memudahkan kolaborasi antara tim desainer dan developer.
4. Dukungan library dan referensi yang sangat luas karena sudah digunakan sejak awal perkembangan Android.

Kekurangan:

1. Memerlukan pengelolaan 2 file berbeda. file XML dan file Kotlin untuk satu layar, sehingga kode menjadi lebih tersebar.
2. Penggunaan findViewById() berulang kali untuk setiap komponen rentan terhadap kesalahan seperti salah ID atau null pointer jika ID tidak ditemukan, tetapi ini bisa menggunakan *View Binding*.
3. Pembaruan UI harus dilakukan secara manual melalui *listener* dan fungsi terpisah, sehingga alur data lebih sulit dilacak semakin besar kompleksitas aplikasi.
4. Kode XML cenderung menjadi panjang, berulang, hingga *nested* untuk tampilan yang kompleks.

- Jetpack Compose

Kelebihan:

1. UI dan logika berada dalam satu file Kotlin sehingga lebih mudah dibaca dan dikelola dalam satu konteks.
2. Sistem *state* reaktif dengan *remember* dan *mutableStateOf* memastikan UI selalu sinkron dengan data secara otomatis tanpa pembaruan manual.
3. Pola *state hoisting* membuat alur data lebih terstruktur dan mudah dilacak karena *state* selalu mengalir dari atas ke bawah.
4. Komponen Composable bersifat modular dan sangat mudah digunakan kembali di berbagai layar hanya dengan memanggil fungsinya.

5. Tidak memerlukan findViewById() ataupun *View Binding* sehingga mengurangi risiko kesalahan referensi komponen.

Kekurangan

1. Kurva belajar yang lebih pekat terutama bagi developer yang terbiasa dengan XML karena memerlukan pemahaman konsep baru seperti rekomposisi, *state hoisting*, dan lambda Composable.
2. Beberapa API seperti *ExposedDropdownMenuBox* masih bersifat eksperimental sehingga memerlukan anotasi @OptIn dan berpotensi kegagalan.
3. Preview UI melalui fitur @Preview di Android Studio terkadang tidak seakurat *Layout Editor* pada XML, terutama untuk tampilan yang kompleks atau interaktif.
4. Performa rekomposisi yang tidak dikelola dengan baik dapat menyebabkan UI dirender ulang secara berlebihan jika state tidak dirancang dengan tepat.

- Secara personal, dalam mengerjakan Modul ini saya lebih memilih menyusun UI dalam XML karena jauh lebih mudah dengan *drag* and *drop*, dibandingkan dengan Compose yang harus menyusun dari awal sampe akhir (misalnya *ExposedDropDownMenu*). Tetapi selain itu, saya memilih Compose untuk semuanya, misalnya dalam logika.

SOAL Essay

Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**

Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

A. Fitur yang Menjadi Daya Jual Kotlin

1. Statically Typed + Type Inference

   "Kotlin is a statically typed programming language: The Kotlin compiler can validate that the methods and fields you're trying to access on an object actually exist. This helps eliminate an entire class of bugs, rather than crash at runtime, if a field is missing or the return type of a function call isn't as expected, you will already see these problems at compile time." [1].

   "The ability of the compiler to determine types from context is called type inference. Type inference in Kotlin means most of the extra verbosity associated with static typing disappears, because you don't need to declare types explicitly." [1].

   "Kotlin is statically typed, for example, so you can't perform inappropriate actions on the wrong type of variable and crash your code. And most of the time, you don't even need to explicitly specify the type yourself as the compiler can infer it for you." [2]

   Intinya sih, dalam Kotlin jarang terjadi *Runtime Error* karena *compiler*nya dapat memvalidasi *method* dan *field* yang diakses, ini yang disebut *Statically Typed*.

   Type Inference adalah kemampuan Kotlin untuk otomatis menentukan tipe data tanpa dideklarasi secara eksplisit, ini sangat membantu mengurangi kompleksitas Kotlin yang bersifat *Statically Typed*.

2. Coroutines

   "Kotlin approaches the problem of concurrent and asynchronous programming using suspendable computations called coroutines, where code can suspend its execution, and resume its work at a later point." [1].

   "Coroutines let you write code that's run asynchronously. This means less time hanging around, a better user experience, and it can also make your application more scalable." [2].

   Ini mungkin fitur Kotlin yang paling saya sering bicarakan, karena berguna untuk *background process* secara *async*, yaitu tidak memblokir proses lainnya, seperti *thread* penting yang digunakan untuk UI.

3. Null Safety

"Kotlin strives to remove the NullPointerException from your program. Kotlin's type system tracks values that can and can't be null and forbids operations that can lead to a NullPointerException at runtime." [1]

"A nullable type is one that allows null values. Unlike other programming languages, Kotlin tracks values that can be null to stop you from performing invalid actions on them. Performing invalid actions on null values is the most common cause of runtime problems in other languages such as Java." [2]

Saya rasa ini merupakan fitur utama yang menjadi daya jual Kotlin, misalnya dibanding dengan Java. Kotlin didesain untuk *null-safety*, berarti variabel tidak bisa bernilai null kecuali secara eksplisit diizinkan, dan akses terhadap nilai null harus ditangani dengan cara yang aman oleh compiler untuk mencegah Billion Dollar Mistake nya Java, yaitu *NullPointerException*.

Masih banyak fitur yang menjadi daya jual Kotlin, seperti *smartcast*, *data class*, *first class functions*, *String Templates*, tetapi untuk fitur utama daya jualnya saya rasa ketiga ini cukup karena laporan sudah cukup panjang.

B. Test Case Fitur Unggulan Kotlin

Test case yang saya gunakan adalah penggunaan Null Safety, Lambda, dan Data Class untuk mendapatkan output dari sebuah LLM dengan REST melalui OkHttp.

Tabel 4. Test Case

No	Main.kt
1	`data class User(val name: String, val age: Int?)`
2	
3	`fun getAgeCategory(age: Int?): String {`
4	`    return when (age) {`
5	`        null -> "Unknown"`
6	`        in 0..12 -> "Child"`
7	`        in 13..19 -> "Teen"`
8	`        else -> "Adult"`
9	`    }`
10	`}`
11	
12	`fun main() {`
13	`    val users = listOf(`
14	`        User("Eula", 21),`
15	`        User("Barbatos", null),`

```
16          User("Jean", 22)
17      )
18
19      val results = users.map { user ->
20          val ageText = user.age?.toString() ?: "No age"
21          val category = getAgeCategory(user.age)
22
23          "${user.name} - $ageText - $category"
24      }
25
26      results.forEach { println(it) }
27  }
```

Kode ini dibuat untuk mendemonstrasikan fitur-fitur unggulan Kotlin berupa Null Safety, Lambda, dan Data Class. Secara sederhana, program ini membuat daftar pengguna dengan nama dan usia, mengkategorikan usia setiap user, dan menampilkan hasilnya ke konsol.

'data class User(val name: String, val age: Int?)' adalah data class, jika dalam Java, membuat class seperti ini perlu banyak hal untuk getter, setter, constructor, toString, equals. Jadi ini merupakan salah satu fitur unggul Kotlin.

'val age: Int?' Kotlin normalnya tidak mengizinkan null, tetapi karena ada ? pada Int, maka membuat variabel tersebut bersifat nullable, walau seseorang tidak mungkin tidak memiliki umur, jadi dibuat untuk gambaran saja.

'when (age){..}' berguna selayaknya switch di Java tetapi jauh lebih ekspresif karena mengecek null langsung, menggunakan *range* sebagai kondisi, dan berfungsi sebagai expression. Terdapat juga Safe Call '?.' (hanya memanggil saat tidak null), '?:' (kalau hasilnya null, gunakan nilai default "No. age")

'val results = users.map { user -> ... }', 'results.forEach { println(it) }' adalah *Lambda,* yaitu cara singkat untuk menulis fungsi yang lumayan rumit, membuat kode lebih ringkas dan bersih. Jika kedua baris tersebut tidak menggunakan lambda maka bisa terlihat seperti ini:

Tabel 5. Tidak Menggunakan Lambda

```
1       val numbers = listOf(1, 2, 3)
2
3       val results = mutableListOf<Int>()
4       for (item in numbers) {
5
```

```
        val doubled = item * 2
        results.add(doubled)
    }
    for (it in results) {
        println(it)
    }
```

Jauh lebih rumit dari versi lambdanya, jadi lambda merupakan salah satu fitur kesukaan developer modern karena keringkasannya.

Fitur terakhir yang digunakan adalah *String Templates*, ‘"${user.name} - $ageText - $category"’ yaitu kemampuan untuk menyisipkan variabel langsung ke dalam string menggunakan ‘$’, jauh lebih bersih dari Java yang menggunakan concatenation.

TAUTAN GITHUB

https://github.com/SebastianAbeSantoso/MODUL-MOBILE/tree/main/MODUL%202

DAFTAR PUSTAKA

[1] R. E. S. I. D. J. Sebastian Aigner, “Kotlin in Action, Second Edition MEAP V09,” 2024.

[2] D. Griffiths and D. Griffiths, “Head First Kotlin.”